

JevAny Technical Report -- Agent-Harness Appendix

SimpleJev

Technical report appendix

Evidence snapshot: 30 September 2026 Appendix revision: 1 October 2026 Scope: bounded Jev decision-making inside LLM agents

This appendix extends the released nine-page **JevAny Technical Report**. The original PDF is preserved unchanged. It studies a specific systems question: can an LLM retain task reasoning and completion ownership while delegating bounded, basic decisions to Jev?

A. Executive conclusion

Jev can benefit an LLM agent when the LLM already has a viable task strategy and the remaining work contains repeated, low-risk choices with several real alternatives. In that regime, Jev is a bounded selection layer: the LLM interprets the task, performs difficult edits, diagnoses failures, verifies the result, and decides when the task is complete. Candidate source depends on the protocol: environments or the DOM supply controlled/web actions, while the LLM generates terminal menus. Jev only chooses within the bounded menu it receives.

The benefit is conditional rather than universal. The strongest repeated result is GPT-5.6-sol on FrozenLake: success stayed at 100% while frontier calls, tokens, and wall time fell by 64.4%, 63.1%, and 37.6%. By contrast, Sokoban usually became more expensive, the fixed six-task WebArena aggregate preserved success but used more calls and tokens, and both medium Terminal-Bench tasks failed with and without Jev. Jev cannot repair a missing global plan or add capability that is absent from the frontier model.

The practical objective is therefore not maximum delegation. It is maximum replacement of expensive frontier decisions subject to preserved verifier quality, bounded risk, and positive end-to-end efficiency.

B. System boundary and comparison

The LLM-generated terminal harness exposes the following loop:

1. The frontier LLM reads the task and current observation.
2. It emits a public reasoning summary, subgoal, one to four candidate actions, expected outcomes, and a `frontier_choice` fallback.
3. The frontier is instructed to mark precise edits, irreversible or security-sensitive operations, exception recovery, and completion as non-routine; singleton actions are locally excluded from delegation.
4. For an eligible routine decision, Jev scores only the supplied candidates.
5. The harness checks the returned index and confidence, executes the selected action, and returns the observation to the LLM.
6. The LLM interprets the observation, revises the plan, verifies the artifact, and calls completion. An independent environment verifier determines reward.

Routine eligibility is model-declared, not a semantic safety proof. The local harness enforces schema, menu cardinality, confidence fallback, a short sequence cap, and narrow command checks; it cannot generally determine whether arbitrary shell commands are reversible or safe. Terminal-Bench runs inside an isolated task container, but this protocol is not a general-purpose command sandbox.

Its D0 baseline is LLM-only under the same candidate protocol: the LLM generates the candidates and the harness executes its `frontier_choice`. Within this terminal protocol, the intervention changes who selects eligible actions, not who reasons about the task. It is not an unconstrained text-only shell baseline, and separate runs can still produce different stochastic trajectories.

The repeated cross-domain D2 matrix uses a different integration. FrozenLake, Sokoban, WebShop, and WebArena expose candidates from the environment or DOM. The optional agent adds a delegation tool and its system instructions; the baseline acts directly without that tool. Those paired rows therefore measure the complete optional-harness intervention, including its prompt/tool surface, not a selector-only ablation. Results from the two protocols are reported separately. D2 uses benchmark-sandbox action menus without a semantic risk filter and may delegate terminal UI controls such as `buy now`; it is not a production authorization or safety boundary.

C. Two orthogonal levels

Task openness and Jev autonomy are independent. A hard task can use no delegation, while a controlled task can delegate nearly every action. Results must be indexed by both axes rather than collapsed into one "easy-to-hard" scale.

- Task tier: T0 controlled; Environment property: Small deterministic state/action space; Examples: FrozenLake, Sokoban; Primary stressor: Selection mechanism and paired-seed regressions
- Task tier: T1 bounded simulator; Environment property: Longer plans but finite affordances and re-settable state; Examples: ALFWorld; Primary stressor: Subgoal order and state tracking
- Task tier: T2 structured interactive; Environment property: Open-text query with bounded interface controls; Examples: WebShop; Primary stressor: Search and refinement
- Task tier: T3 dynamic web; Environment property: Heterogeneous live UI state, popups, accounts, navigation; Examples: WebArena fixed sample6; Primary stressor: Robust browser interaction
- Task tier: T4 open terminal; Environment property: Open commands, persistent side effects, diagnosis, editing; Examples: Terminal-Bench 2; Primary stressor: General software/system completion

ALFWorld belongs at T1, not at the top of the ladder: it can require long plans and object-state tracking, but its action grammar and simulator are bounded. T4 is further stratified by a frozen historical frontier pass rate: easy (4/4), medium (2/4), and hard (0/4). These are sampling strata, not post-hoc claims about Jev. ALFWorld has not been run in the current formal matrix, so this is a protocol placement rather than an empirical Jev result.

- Delegation level: D0 direct baseline; Jev role: None; Execution: Execute LLM frontier_choice; Current evidence: Implemented
- Delegation level: D1 shadow judge; Jev role: Score but do not execute; Execution: Log agreement/regret; Current evidence: Proposed
- Delegation level: D2 selective; Jev role: LLM opts in; Execution: One bounded action with fallback; Current evidence: 96-pair matrix
- Delegation level: D3 routine default; Jev role: All eligible basic decisions; Execution: Normally one action; at most two if composable; Current evidence: Terminal-Bench harness
- Delegation level: D4 bounded subgoal; Jev role: Repeated selection from an LLM-authored library; Execution: Stop on novelty, risk, or verification; Current evidence: Proposed

Configured delegation and realized delegation are different quantities. Report both eligible coverage and all-command replacement. Neither is automatically a frontier-call saving: D2 or D3 can still call the LLM before every choice. D4 is the intended design for amortizing one expensive planning call over several safe local decisions, but it has not yet been evaluated.

Decision significance is a third reporting axis:

- Level: A0 cosmetic; Meaning: Paraphrases or operationally equivalent commands; Default policy: Do not delegate; collapse the menu
- Level: A1 diagnostic; Meaning: Different information sources or verification methods; Default policy: Primary Jev target
- Level: A2 path branch; Meaning: Different products, routes, or recoverable local approaches; Default policy: Delegate only with immediate feedback and fallback
- Level: A3 strategic/high risk; Meaning: Architecture, exact edits, credentials, destructive or irreversible actions; Default policy: Keep with the frontier LLM

Most current open-agent evidence is A1, with limited A2 navigation. A high candidate count alone is therefore not evidence of high-level agent autonomy.

D. Practical operating principles

D.1 Delegate only eligible decisions

Delegate when all of the following are true:

- the choice is routine, locally decidable, low risk, and reversible;

- the menu contains at least two valid alternatives with materially different observations or local paths;
- the current observation contains enough information to rank them;
- a wrong choice can be detected and recovered without corrupting the task;
- the cost of Jev selection is lower than another frontier reasoning call.

Inspection, navigation, waiting versus polling, compilation, and choosing a verification method are good candidates. Exact file contents, architectural strategy, destructive commands, security-sensitive actions, and final completion remain with the frontier LLM.

D.2 Design choices, not paraphrases

Candidate sets should be small (normally two to four), executable, and outcome-oriented. "Inspect raw bytes," "query the database schema," and "check available tools" are useful alternatives because they expose different evidence. Three paraphrases of ls are not. Each menu must preserve a safe LLM fallback and should state the expected observation. A routine singleton is not a decision; the final strict protocol rejects it and asks the LLM to regenerate a real menu.

D.3 Batch only state-compatible actions

Batch decisions only when the LLM explicitly marks them composable and an earlier action cannot invalidate the later candidate. The current terminal protocol caps a delegated sequence at two commands and stops on no progress, an incomplete command, novelty, or changed state. Early unlimited sequences were harmful: on the first regex-log prototype, both modes passed, but optional delegation increased frontier calls from 20 to 30 and tokens from 231,070 to 395,501.

D.4 Make fallback a first-class action

Jev may select only a valid candidate index. On malformed output, service failure, or confidence below 0.55, the harness executes the LLM's fallback. This bounds selection risk but does not remove the latency of a failed Jev request. The SQLite strict smoke passed with one low-confidence fallback; four of five eligible menus were actually delegated.

D.5 Respect the capability ceiling

Jev compares options; it does not invent the missing option or recover a task strategy that the LLM did not express. If the frontier cannot formulate the configuration plan, database recovery algorithm, or correct code edit, better local selection is insufficient. This was visible in both medium Terminal-Bench tasks: 74–79% command replacement did not change 0% success and made the runs more expensive.

D.6 Optimize the frontier, not delegation rate

More delegation is useful only if verifier quality survives and total calls, tokens, cost, or time improve. A point is dominated when another point has no worse verifier quality and no greater resource use. Delegation coverage is an annotation, not an objective. Select demonstrations from a frozen sample and show every negative row that informed the choice.

D.7 Measure quality and efficiency together

Every result should include task and delegation tier, protocol version, exact frontier/Jev checkpoints, verifier reward, termination reason, frontier calls, tokens, estimated API cost, wall time, Jev calls/latency, eligible coverage, command replacement, repairs, failures, and fallbacks. Positive "saved" values mean 1 – optional / baseline. Jev GPU serving cost is not monetized in the current results and must remain a separate axis. A one-attempt result is a mechanism demonstration, never a population estimate.

E. When Jev helps and when it does not

- Dimension: Frontier capability; Conditions associated with benefit: LLM has a sound global plan and can interpret observations; Conditions associated with failure or overhead: Missing plan,

wrong edit, or insufficient domain capability; Evidence: SQLite succeeds; both medium TB2 tasks fail

- Dimension: Choice structure; Conditions associated with benefit: Two to four real, locally rankable alternatives; Conditions associated with failure or overhead: Singleton, paraphrases, or an omitted correct action; Evidence: SQLite raw/schema/tools menu; regex v4 had zero realized coverage
- Dimension: Risk and feedback; Conditions associated with benefit: Reversible action with immediate observation; Conditions associated with failure or overhead: Irreversible action or delayed/ambiguous feedback; Evidence: Inspection/navigation work better than precise edits
- Dimension: Repetition; Conditions associated with benefit: Many similar basic decisions, or a safely composable short sequence; Conditions associated with failure or overhead: Every action needs fresh global reasoning; Evidence: FrozenLake wins; WebArena aggregate rarely delegates
- Dimension: Cost balance; Conditions associated with benefit: Frontier call is expensive relative to Jev; Conditions associated with failure or overhead: Task action is already trivial and short; Evidence: FrozenLake benefits; Sokoban token use rises
- Dimension: Environment stability; Conditions associated with benefit: Candidate meaning survives until execution; Conditions associated with failure or overhead: Popups, changing pages, or stateful side effects invalidate candidates; Evidence: Popup-aware smoke works; aggregate WebArena regresses
- Dimension: Evaluation; Conditions associated with benefit: Independent verifier and paired repetitions; Conditions associated with failure or overhead: Self-reported completion or one stochastic trajectory; Evidence: 96-pair matrix versus exploratory TB2 rows

The observed task pattern is correspondingly mixed:

- Successful efficiency use: FrozenLake with GPT-5.6-sol and 27B Jev; bounded direction choices preserve success while replacing costly calls.
- Promising but not confirmed: WebShop moves from 50% to 60% success with small resource savings, but the paired intervals cross zero.
- Useful terminal mechanism: SQLite recovery preserves reward across a three-point delegation-rate sweep while the sampled 100% target uses fewer frontier resources. It remains a single-attempt curve.
- Quality-recovery signal: query-optimize changes from failure to success with slightly fewer tokens and cost, but wall time increases. One run cannot establish causality.
- Overhead-dominated: Sokoban is too cheap and simple to justify the added choice-generation/Jev path.
- Capability-limited: configure-git-webserver and llm-inference-batching-scheduler fail in both conditions despite high Jev command share.
- No aggregate web gain: the frozen six-task WebArena row keeps success at 50% but increases calls, tokens, and estimated cost.

F. Repeated cross-domain matrix

The frozen D2 matrix contains 96 paired comparisons (192 agent episodes) across ten frontier/benchmark/Jev cells. FrozenLake and Sokoban are controlled mechanism tests; WebShop and WebArena are interactive-agent checks. The WebArena row is the predeclared tasks [264, 278, 300, 158, 4, 763], not the full benchmark.

- Frontier: Opus 4.7; Task: FrozenLake; Jev: 4B; Pairs: 10; Success D0→D2: 100%→90%; Calls saved: +55.9%; Tokens saved: +51.4%; Time saved: -14.0%
- Frontier: Opus 4.7; Task: FrozenLake; Jev: 27B; Pairs: 10; Success D0→D2: 100%→90%; Calls saved: +51.5%; Tokens saved: +44.6%; Time saved: -21.8%

- Frontier: Opus 4.7; Task: Sokoban; Jev: 4B; Pairs: 10; Success D0→D2: 100%→100%; Calls saved: -2.1%; Tokens saved: -33.2%; Time saved: -28.7%
- Frontier: Opus 4.7; Task: Sokoban; Jev: 27B; Pairs: 10; Success D0→D2: 100%→100%; Calls saved: -29.5%; Tokens saved: -84.4%; Time saved: -51.0%
- Frontier: Opus 4.7; Task: WebArena; Jev: 27B; Pairs: 6; Success D0→D2: 50%→50%; Calls saved: -5.6%; Tokens saved: -28.2%; Time saved: +0.4%
- Frontier: Opus 4.7; Task: WebShop; Jev: 27B; Pairs: 10; Success D0→D2: 50%→60%; Calls saved: +7.7%; Tokens saved: +2.9%; Time saved: +6.1%
- Frontier: Sonnet 4.6; Task: FrozenLake; Jev: 27B; Pairs: 10; Success D0→D2: 100%→100%; Calls saved: +36.6%; Tokens saved: +34.4%; Time saved: -14.1%
- Frontier: Sonnet 4.6; Task: Sokoban; Jev: 27B; Pairs: 10; Success D0→D2: 90%→90%; Calls saved: -10.1%; Tokens saved: -14.8%; Time saved: +3.8%
- Frontier: GPT-5.6-sol; Task: FrozenLake; Jev: 27B; Pairs: 10; Success D0→D2: 100%→100%; Calls saved: +64.4%; Tokens saved: +63.1%; Time saved: +37.6%
- Frontier: GPT-5.6-sol; Task: Sokoban; Jev: 27B; Pairs: 10; Success D0→D2: 100%→100%; Calls saved: -26.2%; Tokens saved: -122.0%; Time saved: -7.5%

Only GPT-5.6-sol/FrozenLake is a clear multi-metric win: paired bootstrap intervals for call, token, and latency differences exclude zero. Its mean optional-minus-baseline differences (95% paired bootstrap intervals) are -2.9 calls [-4.7, -1.3], -1,929.6 tokens [-3,854.3, -427.7], and -5.55 seconds [-10.69, -0.30]. WebShop's point estimates are favorable, but its paired intervals for calls, tokens, and latency cross zero. Opus/FrozenLake saves model work but loses one success in each cell and is slower; it is not a win.

G. Open-terminal exploration

Terminal-Bench 2 uses Opus 4.7 as the frontier and the 27B Jev checkpoint. The v3 sample contains one stochastic D0/D3 pair for each of six predeclared tasks. It is protocol-development evidence, not a statistically powered benchmark.

- Stratum/task: easy / regex-log; Reward D0→D3: 0→1; Calls: 30→18; Tokens: 186,712→196,687; Time: 287.0s→178.8s; D3 command replacement: 5/10
- Stratum/task: easy / sqlite-db-truncate; Reward D0→D3: 1→1; Calls: 17→15; Tokens: 202,102→222,728; Time: 146.2s→163.8s; D3 command replacement: 9/13
- Stratum/task: medium / configure-git-webserver; Reward D0→D3: 0→0; Calls: 30→30; Tokens: 267,467→308,860; Time: 249.5s→326.0s; D3 command replacement: 23/31
- Stratum/task: medium / llm-inference-batching-scheduler; Reward D0→D3: 0→0; Calls: 30→30; Tokens: 572,053→600,218; Time: 210.6s→302.6s; D3 command replacement: 19/24
- Stratum/task: hard / query-optimize; Reward D0→D3: 0→1; Calls: 30→29; Tokens: 366,138→348,370; Time: 831.1s→937.1s; D3 command replacement: 27/29
- Stratum/task: hard / path-tracing; Reward D0→D3: 0→0; Calls: 30→30; Tokens: 574,798→361,174; Time: 558.0s→339.3s; D3 command replacement: 17/26

Aggregated descriptively, success is 1/6→3/6, with 9.0% calls, 6.0% tokens, 6.9% estimated Bed-rock cost, and 1.5% wall time saved. This aggregate must not be read causally: it combines heterogeneous tasks and only one attempt per cell. The full negative rows remain important. Easy sqlite-db-truncate is slower and more token-intensive in v3; both medium tasks consume more resources and still fail; hard path-tracing becomes cheaper but still does not complete.

The later v4 SQLite mechanism sweep holds verifier reward at 1 in all three single attempts:

- Target delegation: 0%; Actual eligible coverage: 0/2; Calls: 15; Tokens: 202,050; Estimated frontier cost: \$1.292; Time: 187.9s
- Target delegation: 50%; Actual eligible coverage: 1/4; Calls: 11; Tokens: 161,261; Estimated frontier cost: \$1.009; Time: 155.1s

- Target delegation: 100%; Actual eligible coverage: 2/3; Calls: 8; Tokens: 121,293; Estimated frontier cost: \$0.764; Time: 144.7s

Relative to this sampled 0% run, the 100% target reduces calls 46.7%, tokens 40.0%, estimated frontier cost 40.9%, and time 23.0%. The curve is a demo, not a confidence interval. A later strict-protocol (1.2.0) smoke also passes: reward 1, 10 frontier calls, 138,725 tokens, \$0.873 estimated frontier cost, and 91.8 seconds. Jev executes four of five eligible multi-option decisions and seven of ten commands; one low-confidence decision falls back to the frontier.

H. Decision demonstrations

H.1 SQLite recovery: a meaningful three-way decision

In the v4 candidate-logging sweep, the frontier LLM receives a truncated SQLite file and proposes three materially different diagnostic actions:

```
[0] od head
    od -A x -t x1z -v /app/trunc.db | head -80
    expected: raw hexadecimal page structure

[1] sqlite schema
    sqlite3 /app/trunc.db '.schema';
    sqlite3 /app/trunc.db 'SELECT * FROM sqlite_master;';
    expected: logical schema if the header is usable

[2] check tools
    which sqlite3 python3
    expected: available recovery tools
```

The LLM's safe fallback is option 0. Jev independently selects option 0 at 0.81 confidence. The result exposes a 4,096-byte page beginning with byte 0d, ten cells, content start 0x0f49, and ten cell-pointer offsets. The LLM then recognizes a SQLite leaf-table b-tree page, parses cell pointers, varints, and serial types, recovers testword00 through testword09, corrects an output construction error, and writes /app/recover.json. Harbor's independent verifier returns reward 1.

This division of labor is substantive but bounded: Jev chooses among raw-byte, logical-schema, and tool-availability paths; the LLM performs the recovery algorithm and completion verification. A separate targeted Jev return below 0.55 is rejected, and the harness executes the LLM fallback instead.

H.2 WebArena popup-aware navigation: useful but only a smoke

For WebArena task 264 ("browse products in Cabinets, Racks & Shelves"), both frontier-only and optional-Jev runs pass. The optional trace executes hover [887], hover [2641], and click [2661] under Jev control, then the LLM verifies the category and stops. In this one task, browser actions fall 7->4, frontier calls 8->2, tokens 76,674->13,407, and time 96.6->80.4 seconds.

This is a mechanism demo, not aggregate evidence. The frozen six-task WebArena row remains 50%->50% success with 5.6% more calls and 28.2% more tokens. The task 264 trace therefore cannot be used to hide the negative aggregate.

H.3 Query optimization: selection supports, but does not replace, expertise

In the v3 query-optimize run, Jev controls 27 of 29 commands covering schema inspection, data-size checks, duplicate/orphan/null diagnostics, timing, query execution, and output comparison. The LLM authors the decisive CTE/window- function rewrite. The new query reproduces the output and runs in about 0.56 seconds versus about 214 seconds for the original; the independent task reward changes 0->1. Yet agent wall time rises from 831 to 937 seconds. This is an exploratory quality-recovery example, not an end-to-end latency win, and v3 did not persist every unselected candidate text.

H.4 WebShop: a confident local mistake and frontier recovery

WebShop seed 3100 requests a slim-fit short-sleeve men's henley with exact color 155- blue, size x-large, and price below \$40. On an initially plausible product, Jev selects click[light blue] at 0.99

confidence. The environment reports the action as ineffective, and the selected color does not satisfy the exact global constraint. The frontier LLM consumes that observation, returns to search, opens a different product, selects exact 155– blue and x–large, and completes the purchase. The independent verifier returns reward 1.

This trace demonstrates a meaningful action choice and the need for retained frontier ownership. Confidence is not correctness; a local decision model can prefer a plausible near match while missing a task–level constraint. Immediate re–observation lets the LLM recover. The trace stores the executed Jev action but not its complete unselected menu, so it is a recovery demo rather than a fully replayable candidate–menu comparison. The ten–pair WebShop aggregate remains a positive but inconclusive signal because paired intervals cross zero.

I. Failure analysis and protocol evolution

The early results explain why the strict contract matters:

1. Over–delegation: v1 allowed too many commands after one LLM decision.
State changes invalidated later choices and inflated calls and tokens.
2. No amortization: calling the LLM before every basic action and then adding Jev introduces cost without replacing frontier work.
3. Artificial menus: requiring multiple choices for exact edits produced fake alternatives and schema–repair calls. Non–routine exact edits now allow one candidate; routine decisions require at least two.
4. Misclassification: the v4 regex–log sweep labeled all basic actions as non–routine singletons, yielding zero realized Jev coverage. Different configured rates then measured stochastic trajectory variation, not Jev.
5. Capability deficit: high local command replacement cannot rescue a frontier that lacks the global configuration or coding plan.
6. Cheap–task overhead: for short obvious actions, candidate generation, Jev inference, and context transfer cost more than direct execution.
7. Dynamic–state drift: in browsers or persistent terminals, a candidate can become stale after a popup, navigation, or side effect.

The final protocol consequently enforces bounded candidate menus, a short composability horizon, a 0.55 confidence fallback, complete candidate logging, independent verification, and separate configured versus realized coverage. Routine/non–routine semantics remain a frontier–model declaration; the local validator is a narrow policy gate, not a general shell–security boundary.

J. Claims, limitations, and next experiments

Supported claims:

- A Jev decision layer can preserve quality while reducing frontier work on some bounded, repetitive decisions.
- The repeated GPT–5.6–sol/FrozenLake cell demonstrates a controlled multi–metric efficiency win.
- Real agent environments exhibit both positive mechanisms and clear negative results; delegation rate alone does not predict success.

Not yet supported:

- universal agent improvement;
- a causal population–level Terminal–Bench success gain;

- a general WebArena or WebShop win;
- an efficiency claim that includes monetized Jev GPU cost;
- D4 long-horizon autonomous subgoal execution.

The next compact experiment should freeze the six Terminal-Bench tasks and run paired repetitions of D0, D1, and D3 under the final strict protocol. D1 can estimate agreement and counterfactual regret without execution risk. D4 should be attempted only on one easy and one medium task after D3 is stable. Report task-level distributions and paired intervals rather than only aggregate means, and add Jev service utilization/cost, candidate semantic-distance labels, and fallback confidence values. Do not expand the current WebArena sample after observing results; any expansion must be a separately versioned, predeclared study.

K. Reproducibility pointers

- Formal 96-pair matrix: results/agent-harness-v1/formal-matrix.md and .json
- Terminal-Bench v3 snapshot: results/agent-harness-v1/terminal-bench-sample-v3.md and .json
- Delegation-rate sweep: results/agent-harness-v1/terminal-bench-frontier-v4.md and .json
- Decision traces: Sections H.1 (SQLite), H.2 (WebArena), H.3 (query optimization), and H.4 (WebShop)
- Task/delegation protocol: docs/experiments/AGENT_HARNESS_FRONTIER_PROTOCOL.md

Raw sources, failed runs, development versions, exact model IDs, hashes, and selected-command traces are retained. Estimated dollar values use recorded frontier API pricing and exclude Jev GPU rental and energy.